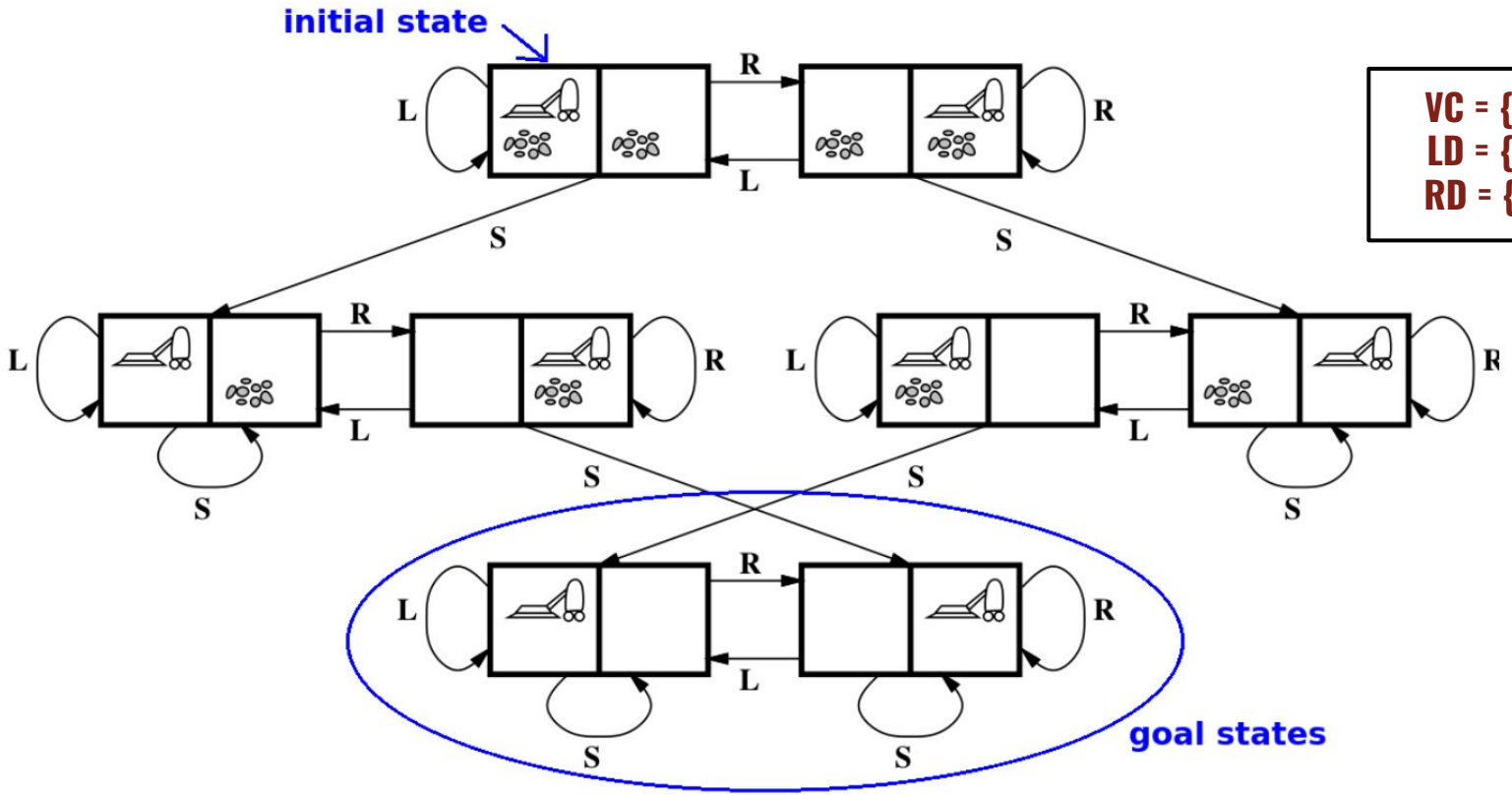
Artificial Intelligence

Search Basics

Prof. Dr. Fazlul Hasan Siddiqui
Office: DUET | **Education**: BSc: IUT; MSc: BUET; PhD: ANU

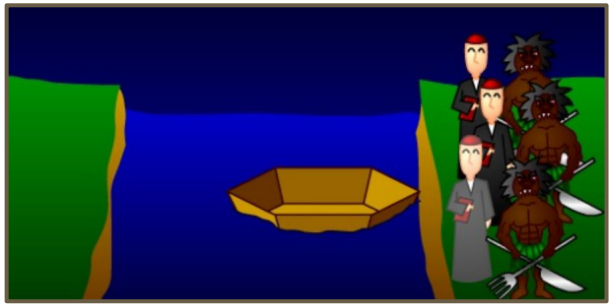
Vacuum Cleaner: State Space

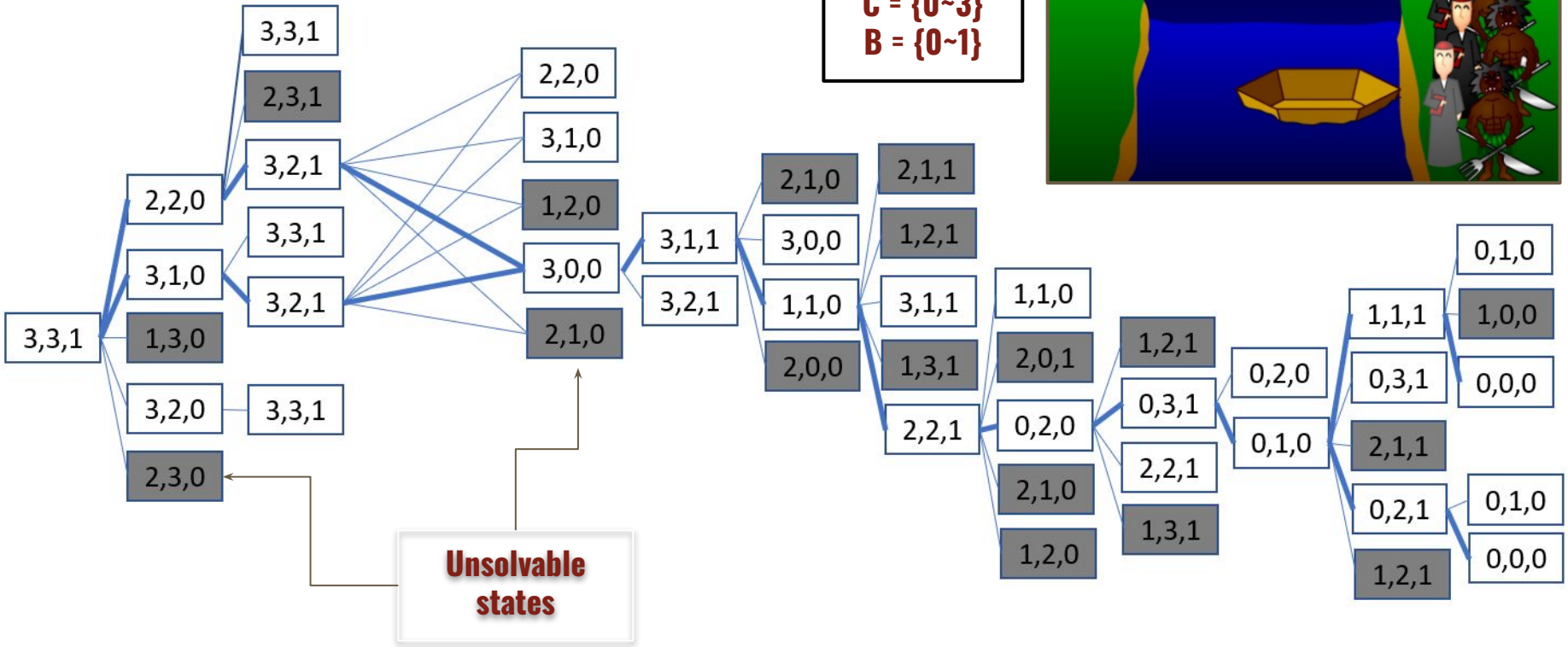


VC = {L, R}
LD = {Y, N}
RD = {Y, N}

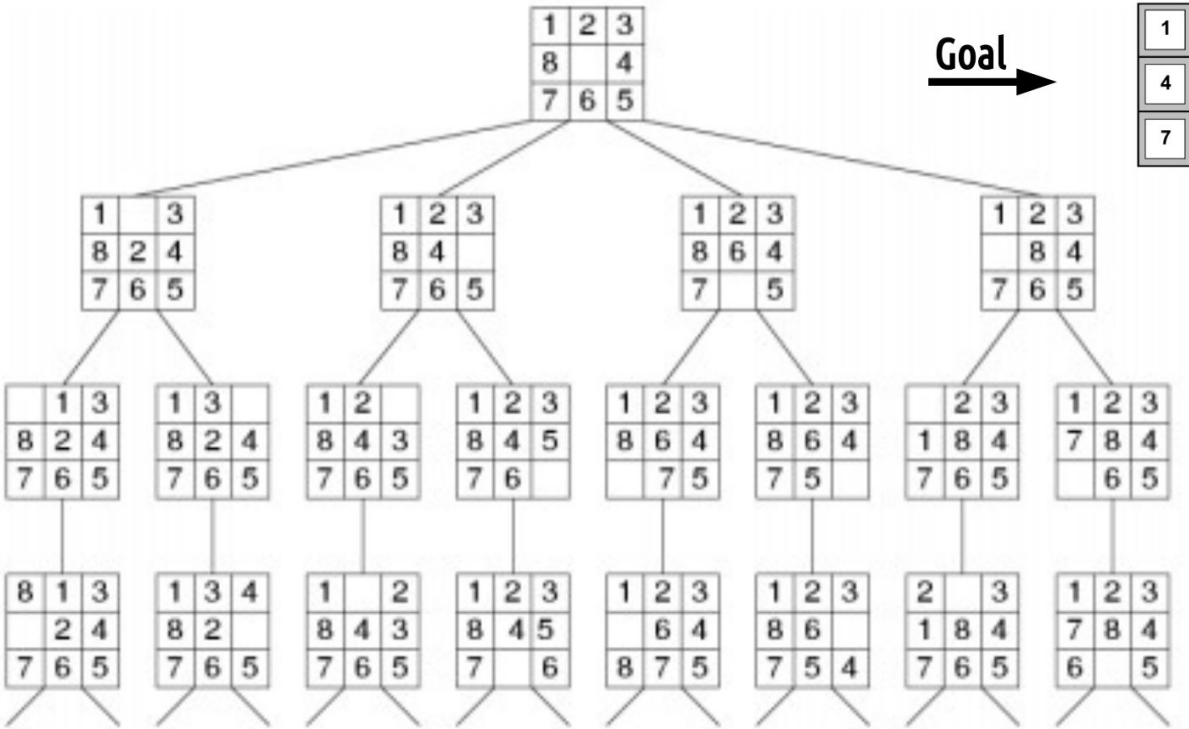
Missionaries and Cannibals: State Space

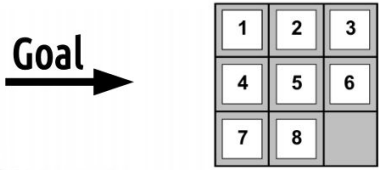
M = {0~3}
C = {0~3}
B = {0~1}





8-Puzzle: State Space



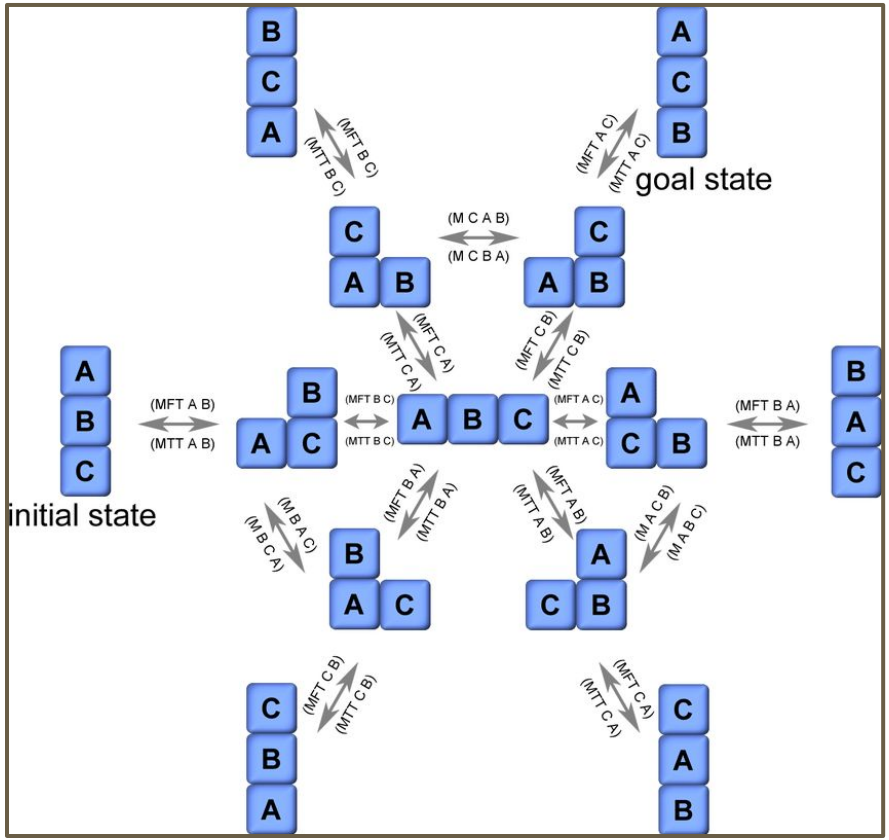


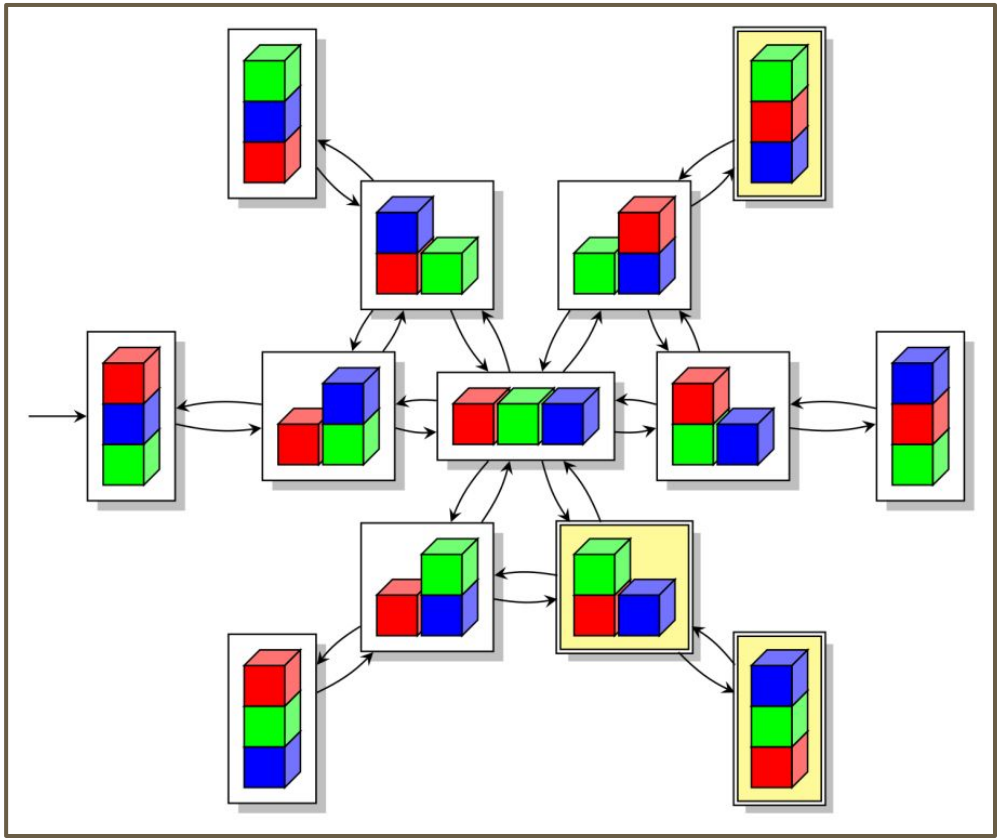
$d = 14$ IDS $=$ 3,473,941 nodes

$d = 24$ IDS $\approx$ 54,000,000,000 nodes

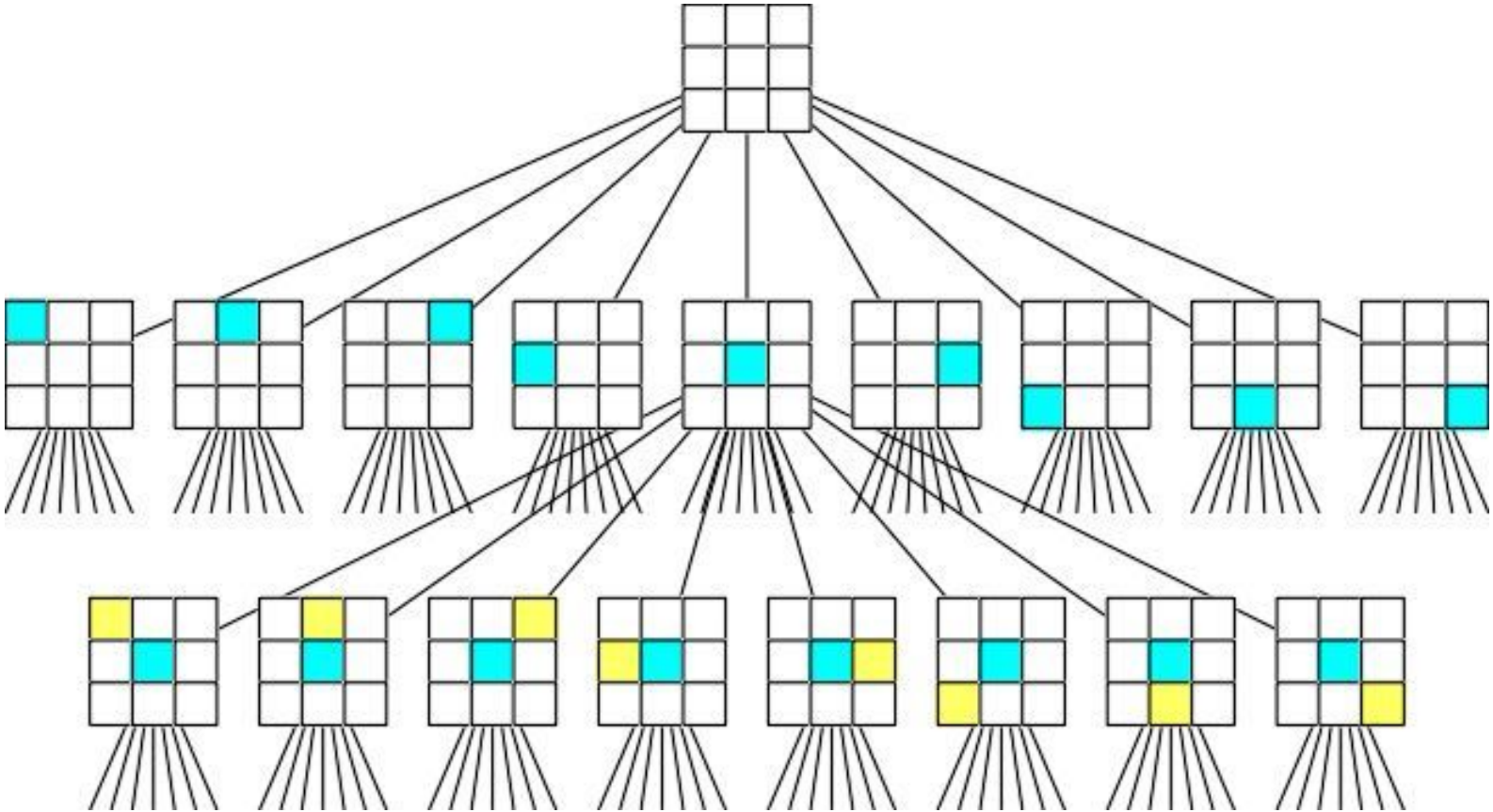
Credit: MIT, USA

Blocks World: State Space

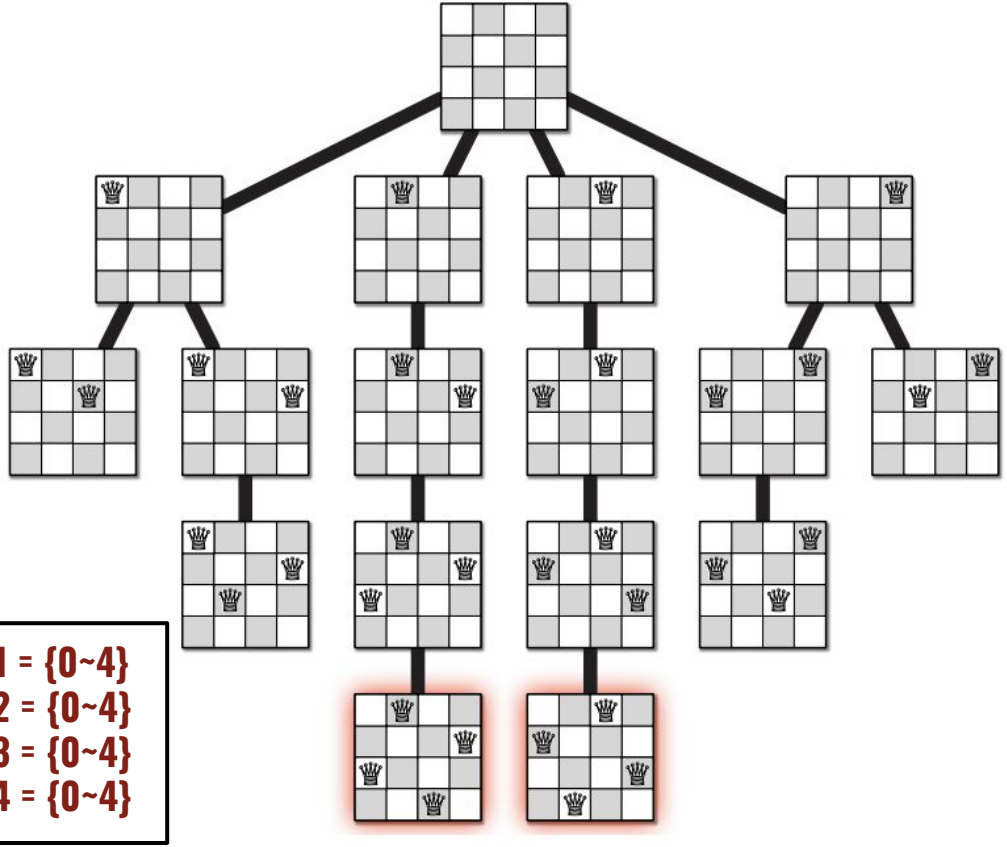




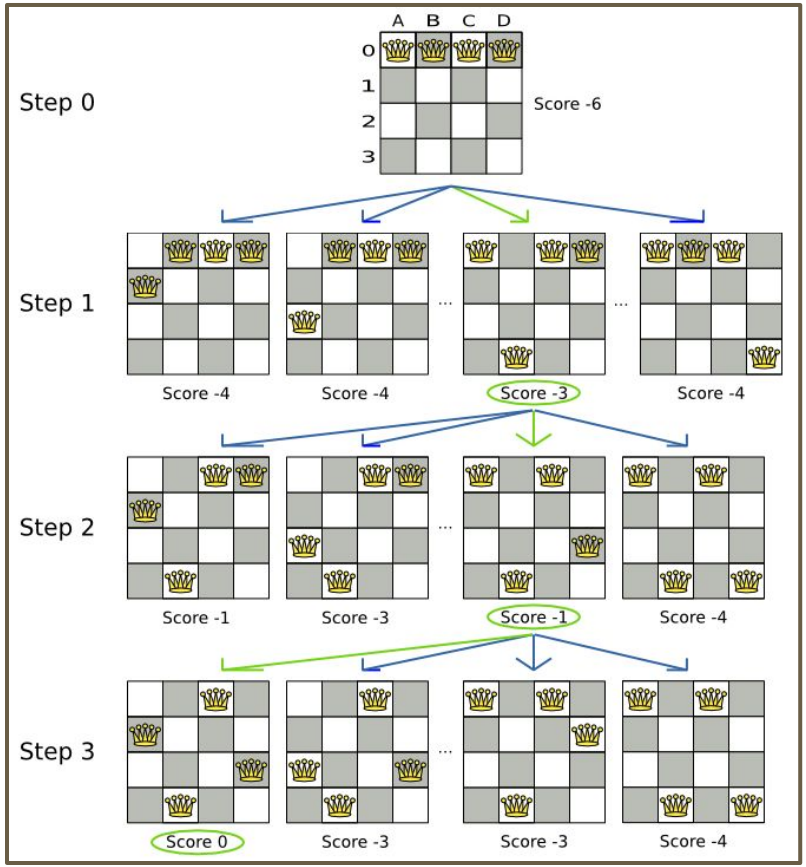
Tic-Tac-Toe: State Space



N-Queen: State Space

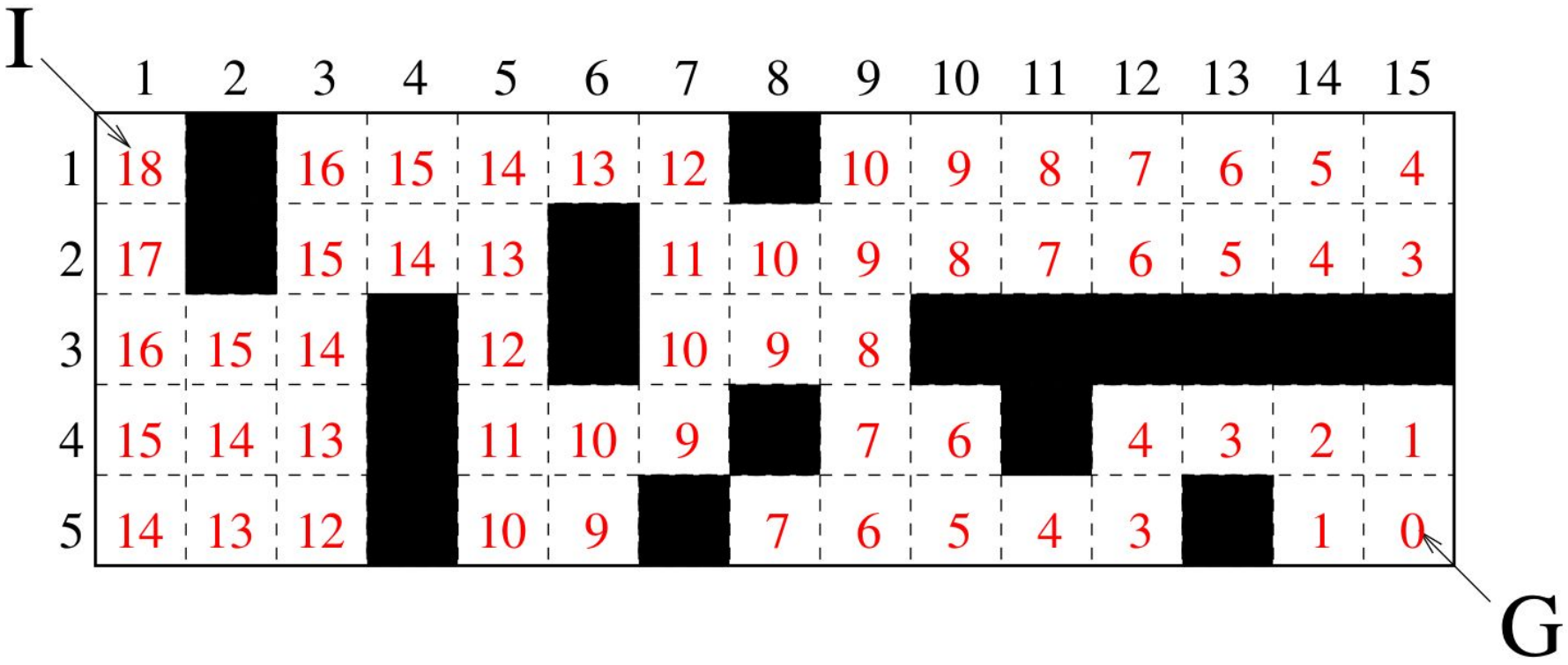


Q1 = {0~4}
Q2 = {0~4}
Q3 = {0~4}
Q4 = {0~4}



Path Planning: Grid Search

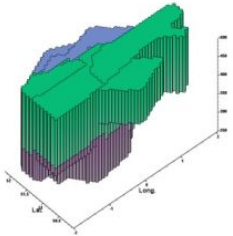
Manhattan Distance, "accurate h":



Planning ...

IBM's Deep Blue wins 6 game match against Garry Kasparov ('97)

Airspace sectorisation



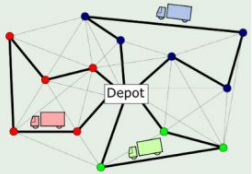
- Go: 2008: MoGo beats Pro (8P) in 9-stone game
- Poker: Next big frontier for AI in games

Example (Vehicle routing: parcel delivery)

Given a depot with parcels for clients and a vehicle fleet, **find** which vehicle visits which client when.

Constraints to be **satisfied**:

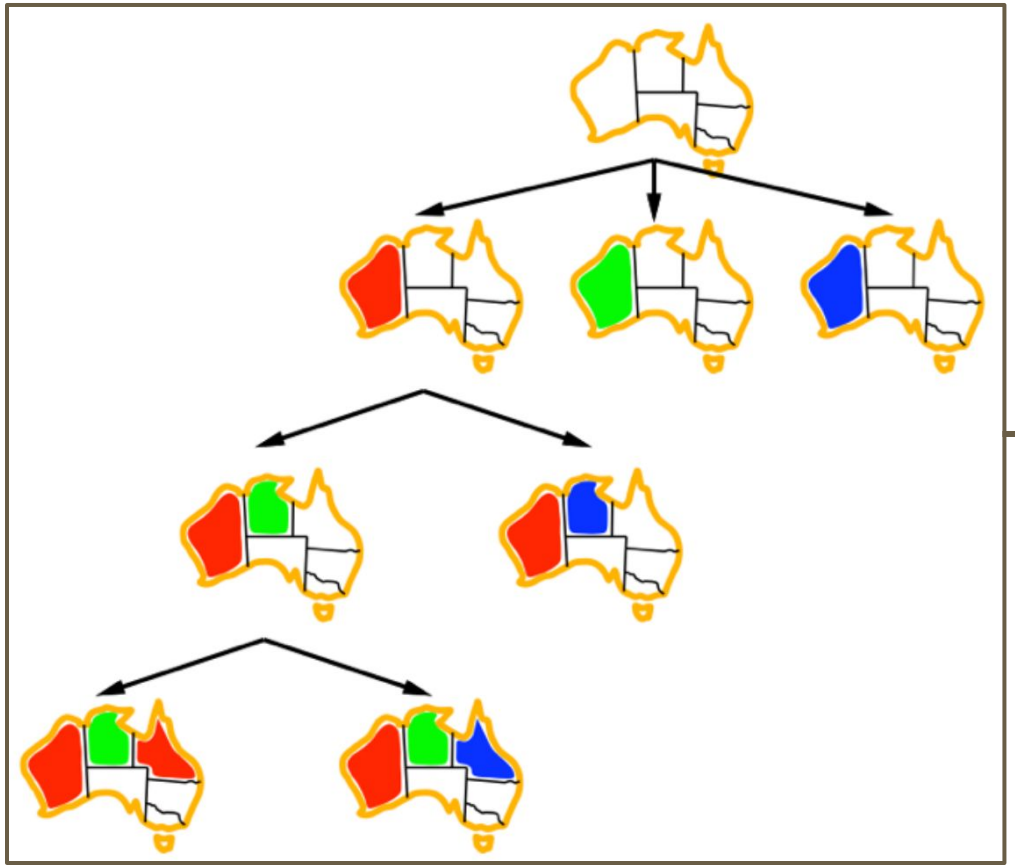
1. All parcels are delivered on time.
2. No vehicle is overloaded.
3. Driver regulations are respected.
4. ...



Objective function to be **minimised**:

- Cost: the total fuel consumption and driver salary.

Scheduling – Map Coloring: State Space



CSP ... (Scheduling)

Example (Agricultural experiment design)

	plot1	plot2	plot3	plot4	plot5	plot6	plot7
barley	✓	✓	✓	–	–	–	–
corn	✓	–	–	✓	✓	–	–
millet	✓	–	–	–	–	✓	✓
oats	–	✓	–	✓	–	✓	–
rye	–	✓	–	–	✓	–	✓
spelt	–	–	✓	✓	–	–	✓
wheat	–	–	✓	–	✓	✓	–

Constraints to be **satisfied**:

1. Equal growth load: Every plot grows 3 grains.
2. Equal sample size: Every grain is grown in 3 plots.
3. Balance: Every grain pair is grown in 1 common plot.

Instance: 7 plots, 7 grains, 3 grains/plot, 3 plots/grain, balance 1.

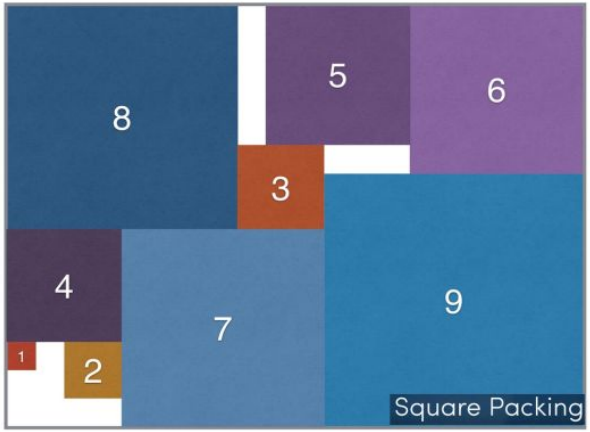
Example (Doctor rostering)

	Mon	Tue	Wed	Thu	Fri	Sat	Sun
Doctor A	call	none	oper	none	oper	none	none
Doctor B	app	call	none	oper	none	none	call
Doctor C	oper	none	call	app	app	call	none
Doctor D	app	oper	none	call	oper	none	none
Doctor E	oper	none	oper	none	call	none	none

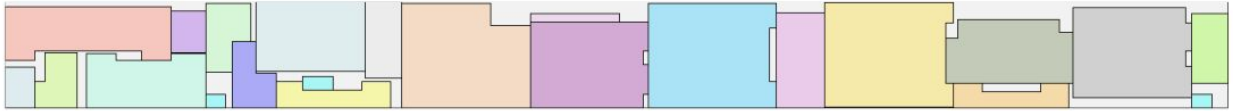
Constraints to be **satisfied**:

1. #doctors-on-call / day $= 1$
2. #operations / workday ≤ 2
3. #operations / week ≥ 7
4. #appointments / week ≥ 4
5. day off after operation day
6. ...

CSP … (Scheduling)



- Complex packing problem
- Room shapes are complex
- Stair carpets have many cutting options
- Pattern of carpet is important

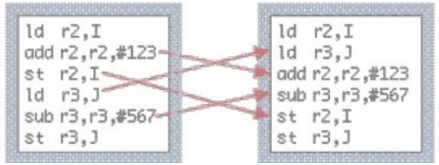


Container packing

School timetabling

COMPILERS FOR INSTRUCTION SCHEDULING

C Compiler
C++ Compiler



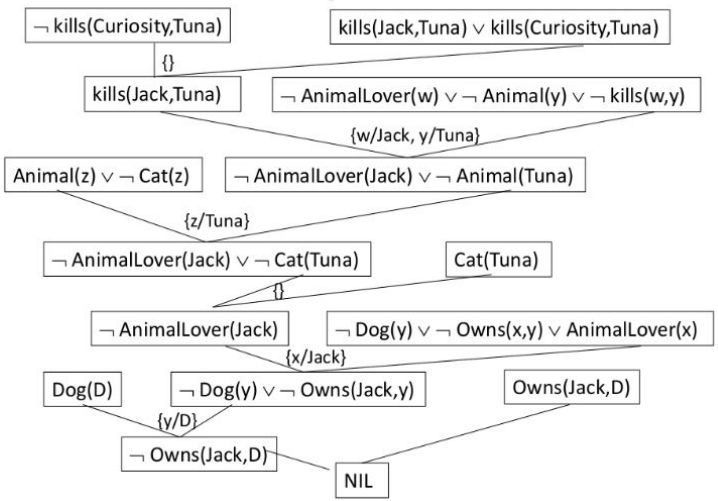
CSP … (Scheduling)

- Jack owns a dog.
- Every dog owner is an animal lover.
- No animal lover kills an animal.
- Either Jack or Curiosity killed the cat, who is named Tuna.
- Did Curiosity kill Tuna?

FOL Representation

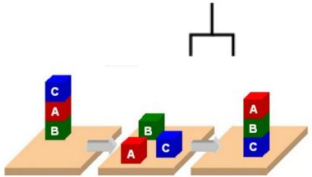
A. $(\exists x)\ (Dog(x) \wedge Owns(Jack,x))$
B. $(\forall x)\ ((\exists y)\ (Dog(y) \wedge Owns(x, y))) \rightarrow AnimalLover(x))$
C. $(\forall x)\ (AnimalLover(x) \rightarrow ((\forall y)\ Animal(y) \rightarrow \neg Kills(x,y)))$
D. $Kills(Jack,Tuna) \vee Kills(Curiosity,Tuna)$
E. $Cat(Tuna)$
F. $(\forall x)\ (Cat(x) \rightarrow Animal(x))$
G. $Kills(Curiosity, Tuna)$ ← Goal

Solving by reasoning (resolution)

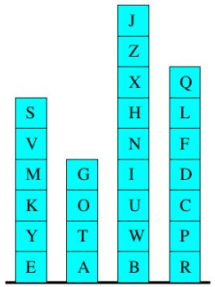


What's the difficulty?

Find the shortest sequence of actions that transforms an **arbitrary** initial configuration of blocks into another **arbitrary** goal configuration

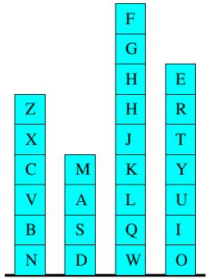


- n blocks, 1 hand.
- A single action either takes a block with the hand or puts a block we're holding onto some other block/the table.



Initial State

blocks	states	blocks	states
1	1	9	4596553
2	3	10	58941091
3	13	11	824073141
4	73	12	12470162233
5	501	13	202976401213
6	4051	14	3535017524403
7	37633	15	65573803186921
8	394353	16	1290434218669921



Goal State

→ State spaces typically are huge even for simple problems.

→ In other words: Even solving "simple problems" automatically (without help from a human) requires a form of intelligence. With blind search, even the largest super-computer in the world won't scale beyond 20 blocks!

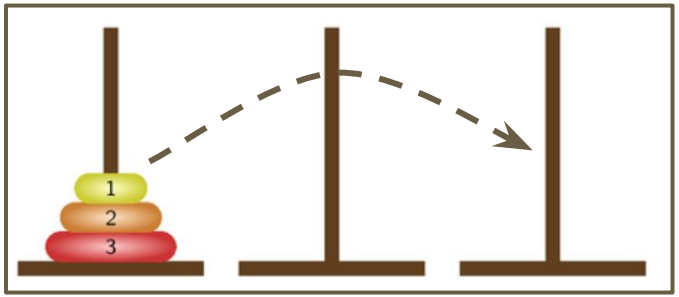
What have all this in common?

- Large search space
- well-identified “goal”
  - Notion of solution is easy to define (declaratively)
- But we don’t know how to reach it
- No algorithm to build a solution incrementally
- Hence:
  - need to explore the search space
  - Either exhaustively or in an “intelligent”, “guided” manner

PDDL (Tower of Hanoi)

domain

```
(define (domain hanoi)
(:predicates (clear ?x)
             (on ?x ?y)
             (smaller ?x ?y))
(:action move
:parameters (?disc ?from ?to)
:precondition (and (smaller ?disc ?to)
                   (on ?disc ?from)
                   (clear ?disc)
                   (clear ?to))
:effect  (and (clear ?from)
              (on ?disc ?to)
              (not (on ?disc ?from))
              (not (clear ?to)))))
```



problem

```
(define (problem hanoi-3)
(:domain hanoi)
(:objects peg1 peg2 peg3 d1 d2 d3 )
(:init
(smaller d1 peg1)
(smaller d2 peg1)
(smaller d3 peg1)
(smaller d1 peg2)
(smaller d2 peg2)
(smaller d3 peg2)
(smaller d1 peg3)
(smaller d2 peg3)
(smaller d3 peg3)
(smaller d1 d2)
(smaller d1 d3)
(smaller d2 d3)
(clear peg2)
(clear peg3)
(clear d1)
(on d3 peg1)
(on d2 d3)
(on d1 d2)
)
(:goal
(and
(on d3 peg3)
(on d2 d3)
(on d1 d2))))
```

plan

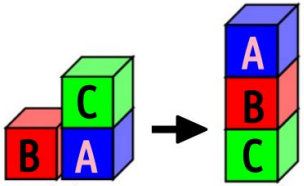
- (move d1 d2 peg3)
- (move d2 d3 peg2)
- (move d1 peg3 d2)
- (move d3 peg1 peg3)
- (move d1 d2 peg1)
- (move d2 peg2 d3)
- (move d1 peg1 d2)

PDDL (Blocks World)

Domain.pddl

```
(define (domain blocks-world-domain)
  (:requirements :strips :equality :conditional-effects)
  (:constants Table)
  (:predicates (on ?x ?y)
               (clear ?x)
               (block ?b)
               )

  (:action move
          :parameters (?obj ?src ?dst)
          :precondition (and (on ?obj ?src) (clear ?obj) (clear ?dst)
                  (not (= ?dst ?src)) (not (= ?obj ?src))
                  (not (= ?obj ?dst)) (not (= ?obj Table)))
          :effect
          (and (on ?obj ?dst) (not (on ?obj ?src))
            (when (not (= ?src Table)) (clear ?src))
            (when (not (= ?dst Table)) (not (clear ?dst))))))
```



init **goal**

Problem.pddl

```
(define (problem sussman-anomaly)
    (:domain blocks-world-domain)
  (:objects A B C)
  (:init (block A) (block B) (block C) (block Table)
     (on C A) (on A Table) (on B Table) (clear C) (clear B) (clear Table))
  (:goal (and (on B C) (on A B))))
```

Solution

1) move C A Table
2) move B Table C
3) move A Table B

PDDL (Knight Tour)

Domain.pddl

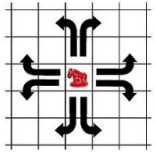
```
(define (domain knights-tour)
  (:requirements :negative-preconditions)

  (:predicates
    (at ?col ?row)
    (visited ?col ?row)
    (diff_by_one ?x ?y)
    (diff_by_two ?x ?y)
  )

  (:action move_2col_1row
   :parameters (?from_col ?from_row ?to_col ?to_row)
   :precondition (and (at ?from_col ?from_row)
                      (diff_by_two ?from_col ?to_col) ; col +/- 2
                      (diff_by_one ?from_row ?to_row) ; row +/- 1
                      (not (visited ?to_col ?to_row)))
   :effect (and (not (at ?from_col ?from_row))
                (at ?to_col ?to_row)
                (visited ?to_col ?to_row))
  )

  (:action move_2row_1col
   :parameters (?from_col ?from_row ?to_col ?to_row)
   :precondition (and (at ?from_col ?from_row)
                      (diff_by_two ?from_row ?to_row) ; row +/- 2
                      (diff_by_one ?from_col ?to_col) ; col +/- 1
                      (not (visited ?to_col ?to_row)))
   :effect (and (not (at ?from_col ?from_row))
                (at ?to_col ?to_row)
                (visited ?to_col ?to_row))
  )
)
```

♞	14	9	20	3
24	19	2	15	10
13	8	23	4	21
18	25	6	11	16
7	12	17	22	5



Problem.pddl

```
(define (problem knights-tour-problem-5x5)
  (:domain knights-tour)

  ; Define a set of "numbers" 1..8:
  (:objects n1 n2 n3 n4 n5)

  (:init
    ; Initial position of the K
    ; (upper left corner):
    (at n1 n5)
    (visited n1 n5)

    ; Here, we have to list all
    ; of the static predicates:
    (diff_by_one n1 n2)
    (diff_by_one n2 n1)
    (diff_by_one n2 n3)
    (diff_by_one n3 n2)
    (diff_by_one n3 n4)
    (diff_by_one n4 n3)
    (diff_by_one n4 n5)
    (diff_by_one n5 n4)

    (diff_by_two n1 n3)
    (diff_by_two n3 n1)
    (diff_by_two n2 n4)
    (diff_by_two n4 n2)
    (diff_by_two n3 n5)
    (diff_by_two n5 n3)
  )

  (:goal (and (visited n1 n1)
              (visited n1 n2)
              (visited n1 n3)
              (visited n1 n4)
              (visited n1 n5)
              (visited n2 n1)
              (visited n2 n2)
              (visited n2 n3)
              (visited n2 n4)
              (visited n2 n5)
              (visited n3 n1)
              (visited n3 n2)
              (visited n3 n3)
              (visited n3 n4)
              (visited n3 n5)
              (visited n4 n1)
              (visited n4 n2)
              (visited n4 n3)
              (visited n4 n4)
              (visited n4 n5)
              (visited n5 n1)
              (visited n5 n2)
              (visited n5 n3)
              (visited n5 n4)
              (visited n5 n5)))
)
```

Picat vs MiniZinc (CSP)

Picat

```
import cp.

go =>
        LD = [F,O,R,T,U,W],
        LD :: 0..9,

        T #>= 1,
        F #>= 1,

        all_different(LD),

                    100 * T + 10 * W + O
                  + 100 * T + 10 * W + O
      #= 1000 * F + 100 * O + 10 * U + R,

        solve(LD),

        writeln(LD).
```

MiniZinc

```
include "alldifferent.mzn";

int: n=9;
var 0..n: T;
var 0..n: W;
var 0..n: O;
var 0..n: F;
var 0..n: U;
var 0..n: R;

constraint          100 * T + 10 * W + O
                  + 100 * T + 10 * W + O
        = 1000 * F + 100 * O + 10 * U + R;

constraint F=1;
constraint alldifferent([T,W,O,F,U,R]);

solve satisfy;

output ["   ",show(T),show(W),show(O),"\n",
        "+  ",show(T),show(W),show(O),"\n",
        "=  ",show(F),show(O),show(U),show(R),"\n"];
```

MiniZinc (Plot Schedule)

	plot1	plot2	plot3	plot4	plot5	plot6	plot7
barley	✓	✓	✓	–	–	–	–
corn	✓	–	–	✓	✓	–	–
millet	✓	–	–	–	–	✓	✓
oats	–	✓	–	✓	–	✓	–
rye	–	✓	–	–	✓	–	✓
spelt	–	–	✓	✓	–	–	✓
wheat	–	–	✓	–	✓	✓	–

Constraints to be **satisfied**:

1. Equal growth load: Every plot grows 3 grains.
2. Equal sample size: Every grain is grown in 3 plots.
3. Balance: Every grain pair is grown in 1 common plot.

Instance: 7 plots, 7 grains, 3 grains/plot, 3 plots/grain, balance 1.

AED.mzn

```
int: nbrBlocks;
int: nbrVarieties;
set of int: Blocks = 1..nbrBlocks;
set of int: Varieties = 1..nbrVarieties;
int: blockSize;
int: sampleSize;
int: balance;
array[Varieties, Blocks] of var 0..1: BIBD;
solve satisfy;

constraint forall(b in Blocks)
(blockSize = sum(v in Varieties)(BIBD[v, b]));

constraint forall(v in Varieties)
(sampleSize = sum(b in Blocks)(BIBD[v, b]));

constraint forall(v, w in Varieties where v < w)
(balance = sum(b in Blocks)(BIBD[v, b] * BIBD[w, b]));

output[show(BIBD[v,w]) ++ if w == nbrBlocks then "\n"
  else " " endif | v in Varieties, w in Blocks]
```

AED.dzn

```
nbrBlocks = 7;
nbrVarieties = 7;
blockSize = 3;
sampleSize = 3;
balance = 1;
```

solution

```
Finished in 60msec
Compiling aed-array.mzn
Running aed-array.mzn
1 0 1 1 0 0 0
0 0 0 1 1 1 0
1 0 0 0 0 1 1
0 0 1 0 1 0 1
0 1 0 1 0 0 1
0 1 1 0 0 1 0
1 1 0 0 1 0 0
----------
```

MiniZinc (Happiness Matrix)

Four male celebrities are marooned on a desert island with five female celebrities. The 'happiness matrix' indicate how much happiness each couple would experience if the spend all their time together. The happiness earned by a couple is proportional to the fraction of time the spend together. For example, if Jon and Jolie spend half their time together, they earn happiness of (1/2)x(14) = 7. The optimal solution requires that each person spend all their time with one person of the opposite sex, so this result is often referred to as the Marriage Theorem.

The goal of the eight celebrities is to maximize the total happiness of the celebrities on the island. Determine the marriage partner for each celebrity.

Happiness	Jolie	Taylor	Kirsten	Lauren	Emma
Jon	14	5	8	7	15
Owen	2	12	6	5	3
Pachino	7	8	3	9	7
Tom	2	4	6	10	1

```
total_cost: 44
0  0  0  0  1
0  1  0  0  0
1  0  0  0  0
0  0  0  1  0
----------
==========
Finished in 97msec
```

solution

Maximize the total happiness of the celebrities

```
int: rows = 4;
int: cols = 5;
array[1..rows, 1..cols] of var 0..1: x;
array[1..rows, 1..cols] of int:  cost;
var int: total_cost =
  sum(i in 1..rows, j in 1..cols) (x[i,j]*cost[i,j]);

% maximize total_cost
solve maximize total_cost;

% exacly one assignment per row
constraint
 forall(i in 1..rows) (
        sum(j in 1..cols) (x[i,j]) = 1
 );

% zero or one assignments per column
constraint
  forall(j in 1..cols) (
        sum(i in 1..rows) (x[i,j]) <= 1
  );

% happiness table: cost
cost =
[|14,  5, 8,  7, 15,
 | 2, 12, 6,  5,  3,
 | 7,  8, 3,  9,  7,
 | 2,  4, 6, 10,  1
 |];
```